

HW2

1. String vs StringBuilder vs StringBuffer

Answer: String is immutable, making it inefficient for frequent modifications because it creates new objects each time. In contrast, StringBuilder and StringBuffer are mutable, where StringBuilder is faster but non-thread-safe, while StringBuffer is thread-safe due to synchronized methods.

2. Comparator vs Comparable, when to use which one

Answer: Comparable defines the natural, inherent ordering of a class by implementing the compareTo method directly inside that class. Conversely, Comparator defines a custom, external sorting strategy without altering the original class code, which is ideal for flexible or multiple sorting criteria.

3. Overriding vs Overloading

Answer: Overloading occurs within the same class where multiple methods share the same name but have different parameter signatures (compile-time polymorphism). Overriding happens between a subclass and a superclass, where the subclass provides a specific implementation for a method with the exact same signature (runtime polymorphism).

4. JRE vs JDK vs JVM

Answer: JVM is the virtual machine that executes Java bytecode, while JRE packages the JVM along with core class libraries to run Java applications. JDK is a full development kit that includes the JRE plus development tools like the compiler (javac) to create Java programs.

5. Java 8 basic data types

Answer: Java 8 has eight primitive data types: byte, short, int, long, float, double, char, and boolean. They are predefined by the language, hold their values directly on the stack, and are not objects.

6. Primitive type vs Reference type

Answer: Primitive types store actual values directly on the stack memory and have no methods or attributes. Reference types, such as classes, interfaces, and arrays, store memory addresses (pointers) on the stack that point to the actual object instances allocated on the heap.

7. How does JVM work

Answer: The JVM works by loading .class bytecode files using the ClassLoader, allocating memory in the Runtime Data Areas, and executing instructions via the Execution Engine. It uses an interpreter to run bytecode line-by-line and a JIT compiler to optimize hot code into native machine code for maximum performance.

8. JVM memory data model

Answer: The JVM runtime memory is divided into thread-shared areas, which are the Heap for objects and the Method Area/Metaspace for class metadata, and thread-private areas, including

the JVM Stack for local variables, the Native Method Stack, and the Program Counter Register.

9. How does GC work

Answer: Garbage Collection works by using the Reachability Analysis algorithm to trace paths from "GC Roots"; any object that is disconnected from these roots is considered unreachable garbage. Once identified, the garbage collector automatically reclaims that memory using specific algorithms like Copying, Mark-Sweep, or Mark-Compact.

10. Young/Old/Perm generation (Metaspace)

Answer: The Heap is split into the Young Generation for newly created, short-lived objects (managed by Minor GC) and the Old Generation for long-lived objects promoted after surviving multiple GC cycles (managed by Major/Full GC). The Permanent Generation was replaced in Java 8 by Metaspace, which stores class metadata in native off-heap memory to avoid heap OOM errors.

11. Different types of GC

Answer: GC types have evolved from single-threaded Serial and throughput-optimized Parallel collectors to low-latency concurrent collectors like CMS. Modern JVMs default to G1 or use ZGC, which divide the heap into regions and perform concurrent, region-based garbage collection to minimize Stop-The-World (STW) pauses.